

Infraestructura como Código

OpenTofu, Terraform y Ansible

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 PI · Puesta en Producción de Aplicaciones

PI · INFRAESTRUCTURA COMO CÓDIGO

01

Evolución de la gestión de infraestructura

Del despliegue tradicional al cloud

Despliegue tradicional

EL PROCESO CLÁSICO

- Comprar el servidor o crear la máquina virtual
- Instalar y configurar el sistema operativo
- Instalar y configurar los servicios
- Configurar la seguridad
- Desplegar la aplicación
- **Documentar... ¿o no?**

PROBLEMAS DEL ENFOQUE TRADICIONAL

- **Configuración artesanal** — difícil de reproducir
- La documentación **se desactualiza** o directamente no existe
- **Escalado vertical**: paradas de servicio
- Misma configuración usada durante años sin revisión
- *"Funciona en mi máquina"*

Despliegue moderno con Cloud Computing

NUEVAS POSIBILIDADES

- Aparición del **Cloud Computing (IaaS)**: AWS, Azure, GCP, OpenStack...
- Aprovisionamiento de VMs **desde una imagen o plantilla**
- Uso de herramientas de **gestión de la configuración**
- Despliegue desde un entorno de pruebas **idéntico al de producción**
- **Escalado horizontal** sin paradas

EL CAMBIO DE MENTALIDAD

- Los servidores no tienen por qué mantenerse con la misma configuración durante años
- La infraestructura es **desechable y reproducible**
- Los recursos se crean y destruyen según la demanda
- **Todo el proceso puede automatizarse**

PI · INFRAESTRUCTURA COMO CÓDIGO

02

Infraestructura como Código

laC: orquestación, configuración y DevOps

¿Podemos programar la infraestructura?

" *Los servicios de virtualización y Cloud Computing son **software**, por lo tanto los podemos **programar** mediante **APIs**.*

SOFTWARE DE ORQUESTACIÓN

Permite **crear y gestionar escenarios** completos: máquinas virtuales, redes, almacenamiento...

Ejemplos: Terraform, OpenTofu, Pulumi, AWS CloudFormation, Heat (OpenStack)

SOFTWARE DE GESTIÓN DE LA CONFIGURACIÓN (CMS)

Permite **configurar el software** de las máquinas que hemos creado.

Ejemplos: Ansible, Puppet, Chef, Salt

Software de Orquestación

Utilizado para crear escenarios completos con **múltiples servidores o contenedores** (aprovisionamiento de recursos).

- Muy útil con **demanda variable de recursos**
- Muy útil cuando la configuración **cambia continuamente**
- Puede incluir funcionalidad de **autoescalado**
- Puede incluir respuestas a **eventos**

ENFOQUE DECLARATIVO

En lugar de describir *cómo* crear la infraestructura, **declaramos el estado deseado** y la herramienta se encarga de alcanzarlo.

```
resource "openstack_compute_instance_v2" "web" {  
    name      = "servidor-web"  
    image_id  = "ubuntu-22.04"  
    flavor_id = "m1.small"  
}
```

Software de Gestión de la Configuración (CMS)

Proporciona la gestión e instalación del software de forma **declarativa**:

AUTOMATIZACIÓN

Instala y configura software de forma **automática** en las máquinas de la infraestructura

DECLARATIVO

Se usan **ficheros de texto** donde se declara la configuración deseada, no los pasos para llegar a ella

IDEMPOTENCIA

Se puede ejecutar una declaración **múltiples veces** y el resultado siempre es el mismo

Cambio de paradigma: Infraestructura como Código

" *Trata tu infraestructura como el **software** que es.*

BUENAS PRÁCTICAS DE DESARROLLO

- **Control de versiones** (Git) para toda la infraestructura
- Código **legible y documentado**
- **Revisión de código** antes de aplicar cambios
- **Tests** para validar la configuración

BENEFICIOS OPERATIVOS

- Escenarios **replicables** y predecibles
- **Auditoría** completa de cambios
- **Recuperación ante desastres** mucho más rápida
- Entornos de desarrollo **idénticos a producción**

DevOps

Conflicto histórico: los equipos de Desarrollo (DEV) y Sistemas (OPS) han tenido objetivos y responsabilidades diferentes.

¿CÓMO SOLUCIONARLO?

- **Mismas herramientas** para dev y ops
- Extender las buenas prácticas de desarrollo a sistemas
- De integración continua → **entrega continua** → **despliegue continuo**
- **Infraestructura como código**
- **Escenarios replicables y automatización** de la configuración

EL OBJETIVO ES COMÚN

Entregar software **de calidad**, de forma **rápida** y **fiable**.

La IaC es una de las piedras angulares de la cultura DevOps.

PI · INFRAESTRUCTURA COMO CÓDIGO

03

Terraform y OpenTofu

Orquestación de infraestructura declarativa

¿Qué es Terraform?

" *Terraform es una herramienta de orquestación de infraestructura de código abierto creada por HashiCorp en 2014.*

CARACTERÍSTICAS PRINCIPALES

- Lenguaje declarativo propio: **HCL** (*HashiCorp Configuration Language*)
- Funciona con **múltiples proveedores**: AWS, Azure, GCP, OpenStack...
- Gestiona el **estado de la infraestructura** en un fichero `terraform.tfstate`
- **Planificación previa** antes de aplicar cambios

EL PROBLEMA: CAMBIO DE LICENCIA

En **agosto de 2023**, HashiCorp cambió la licencia de Terraform de **MPL 2.0** (software libre) a **BSL 1.1** (licencia de negocio), restringiendo su uso comercial por competidores.

Esto impulsó la creación de una alternativa **100% libre**.

¿Qué es OpenTofu?

*"**OpenTofu** es un fork de Terraform mantenido por la **Linux Foundation** bajo licencia **MPL 2.0**. Es el sucesor libre y abierto de Terraform.*

HISTORIA

- Nace en **septiembre de 2023** como respuesta al cambio de licencia de Terraform
- Lo impulsan empresas como Gruntwork, Spacelift, Harness, env0...
- Se convierte en proyecto de la **Linux Foundation** en enero de 2024
- Mantiene **compatibilidad total** con

¿POR QUÉ OPENTOFU?

- **Software libre** (MPL 2.0) sin restricciones de uso
- Comunidad abierta y gobernanza transparente
- Compatible con todos los **proveedores y módulos** de Terraform
- Desarrollo activo con nuevas funcionalidades propias

Conceptos fundamentales de OpenTofu

PROVIDERS

Plugins que permiten a OpenTofu interactuar con distintas **APIs** (AWS, OpenStack, Docker, Kubernetes...).

```
terraform {  
  required_providers {  
    openstack = {  
      source = "terraform-provider-openstack/openstack"  
    }  
  }  
}
```

RESOURCES

Representan los **elementos de infraestructura** que queremos crear o gestionar.

```
resource "openstack_compute_instance_v2" "web" {  
  name      = "servidor-web"  
  image_id  = var.image_id  
  flavor_id = "m1.small"  
}
```

Más conceptos de OpenTofu

VARIABLES

Permiten **parametrizar** la configuración para reutilizarla en distintos entornos.

```
variable "image_id" {  
  description = "ID de la imagen base"  
  type        = string  
  default     = "ubuntu-22.04"  
}
```

MÓDULOS

Agrupaciones de recursos reutilizables.
Permiten **estructurar proyectos** complejos y compartir configuraciones.

```
module "red" {  
  source      = "./modules/red"  
  nombre_red  = "mi-red-privada"  
  cidr        = "192.168.1.0/24"  
}
```

El flujo de trabajo de OpenTofu

LOS CUATRO PASOS BÁSICOS

```
# 1. Inicializar — descarga providers y módulos
tofu init

# 2. Planificar — muestra los cambios que se aplicarán
tofu plan

# 3. Aplicar — crea/modifica/destruye la infraestructura
tofu apply

# 4. Destruir — elimina toda la infraestructura gestionada
tofu destroy
```

EL CICLO DE VIDA

1. `init` — prepara el entorno de trabajo
2. `plan` — compara el estado deseado con el actual y propone cambios
3. `apply` — aplica los cambios después de confirmación
4. `destroy` — elimina los recursos cuando ya no son necesarios

"El paso `plan` es clave: permite revisar los cambios antes de aplicarlos, evitando sorpresas."

Estructura de un proyecto OpenTofu

```
mi-proyecto/  
├─ main.tf          # Recursos principales  
├─ variables.tf     # Definición de variables  
├─ outputs.tf       # Valores de salida (IPs, IDs...)  
├─ providers.tf     # Configuración de providers  
├─ terraform.tfvars # Valores de las variables (no subir a git si tiene credenciales)  
└─ modules/  
    └─ servidor/  
        ├─ main.tf  
        └─ variables.tf
```

i El fichero **terraform.tfstate** se genera automáticamente. Contiene el estado real de la infraestructura. En equipos, se recomienda almacenarlo en un **backend remoto**.

OpenTofu con OpenStack — ejemplo completo

```
provider "openstack" {  
  cloud = "openstack" # Usa el fichero clouds.yaml para las credenciales  
}  
  
resource "openstack_networking_network_v2" "red_privada" {  
  name          = "mi-red"  
  admin_state_up = true  
}  
  
resource "openstack_compute_instance_v2" "servidor" {  
  name          = "servidor-web"  
  image_name    = "Ubuntu 22.04"  
  flavor_name   = "m1.small"  
  key_pair      = "mi-clave-ssh"  
  
  network {  
    uuid = openstack_networking_network_v2.red_privada.id  
  }  
}
```

PI · INFRAESTRUCTURA COMO CÓDIGO

04

Ansible

Gestión de la configuración

¿Qué es Ansible?

" ***Ansible** es un software de **gestión de la configuración (CMS)** que permite declarar y automatizar la configuración de máquinas.*

CARACTERÍSTICAS

- Desarrollado principalmente por **Red Hat**
- Escrito en **Python** (disponible en PyPI)
- Arquitectura **push** — sin agentes, solo SSH
- Configuración declarativa en **YAML**
- Primera versión: **2012**

¿POR QUÉ ANSIBLE?

- Sintaxis **sencilla y conocida** (YAML)
- **Sin agentes** — solo necesita SSH
- **Fácil de instalar** y de empezar a usar
- **Comunidad muy activa**
- Repositorio de roles: **Ansible Galaxy**

Conceptos fundamentales de Ansible

MÓDULOS

Funciones predefinidas que realizan una acción concreta: copiar un fichero, instalar paquetes, reiniciar un servicio...

Cada módulo acepta parámetros obligatorios y opcionales.

```
- name: Instalar Apache
  ansible.builtin.apt:
    name: apache2
    state: present
```

INVENTARIO

Fichero donde se definen los **hosts** y **grupos de hosts** que Ansible va a gestionar.

```
[webservers]
servidor1.ejemplo.com
servidor2.ejemplo.com

[dbservers]
db.ejemplo.com
```

Playbooks y Roles

PLAYBOOKS

Ficheros YAML que agrupan **jugadas (plays)**: conjuntos de tareas para conseguir una configuración completa.

```
- name: Configurar servidor web
  hosts: webservers
  become: true
  tasks:
    - name: Instalar Apache
      apt:
        name: apache2
        state: present
    - name: Copiar configuración
      copy:
        src: apache.conf
        dest: /etc/apache2/sites-enabled/
```

ROLES

Permiten **estructurar y reutilizar** los playbooks dividiéndolos por servicio o funcionalidad.

```
roles/
├─ apache2/
│   ├── tasks/main.yml
│   ├── handlers/main.yml
│   ├── templates/
│   ├── files/
│   └─ defaults/main.yml
```

Los roles se comparten en **Ansible Galaxy**:

```
ansible-galaxy install geerlingguy.apache
```

OpenTofu + Ansible: la combinación perfecta

OPENTOFU — ORQUESTACIÓN

Crea y gestiona la **infraestructura**:

- Máquinas virtuales
- Redes y subredes
- Almacenamiento
- IPs flotantes
- Grupos de seguridad

ANSIBLE — CONFIGURACIÓN

Configura el **software** de las máquinas creadas:

- Instalación de paquetes
- Configuración de servicios
- Despliegue de aplicaciones
- Gestión de usuarios
- Actualizaciones del sistema

 OpenTofu puede invocar Ansible automáticamente como **provisioner**, o bien generar el inventario dinámico para que Ansible lo consuma.

PI · INFRAESTRUCTURA COMO CÓDIGO

¡Gracias!

Infraestructura como Código

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 PI